LEXY

ENGINEERING DOCUMENTATION

DOCUMENT 06 / 09

Engineering Principles & Ways of Working

AI Hiring Platform · Intelligence-First

Generated **June 14, 2026** · Confidential — internal use

DOC 06 OF 09 · LEXY ENGINEERING

Lexy — Engineering Principles & Ways of Working

> Engineering documentation set · Doc 6 of 9 How decisions get made, what the guardrails are, and what "done" means here.

1. The five principles (in priority order)

1. **Tenant safety is non-negotiable.** A cross-tenant data leak is existential. Scope every query (`getAllowedTenantIds`), let RLS backstop you, and never treat header presence as auth. When in doubt, over-scope and ask.
2. **Be honest about uncertainty.** A null score is `—`, not `0%`. Low-data means low-confidence, capped. Cold-start fallbacks are deterministic, never random. The product's value is *defensibility*; a confidently-wrong number destroys it.
3. **Fail loud, never silent.** Prefer an explicit error over a silent fallback to placeholder/mocked data. The exceptions are the *deliberately* graceful paths (scoring config, provider adapters) — and those are documented and tested precisely because silent fallback is dangerous.
4. **Governance is built-in, not bolted-on.** AI messaging, fairness, and adverse writes are guarded by build-breaking checks and test suites — not by reviewer memory. If a safety property matters, encode it so it *can't* ship broken.
5. **Make the next engineer faster.** Write the *why* down (memory files + this doc set). Most sharp edges here were paid for once already; document them so they're paid for only once.

2. How decisions get made

- **Small, reversible changes:** just do them, following existing conventions.
- **Architectural / cross-cutting / destructive changes:** align first. Surface the tradeoff, the blast radius, and the rollback story before writing code. Respect the user preference in `replit.md`: *"Ask before making major changes."*
- **Record the reasoning, not just the result.** Any non-obvious decision — a tradeoff, a convention, a constraint learned the hard way — goes into `.agents/memory/` with a `Why:` line. Implementation changelogs do **not** go there (the code and git history already hold those).
- **Respect documented preferences.** `replit.md` "User Preferences" is binding unless it conflicts with a higher-priority instruction. Notably:
  - Off-limits files (do not edit in place without founder/owner sign-off):

- `artifacts/lexy/src/components/intelligence/**` — the UI for the Intelligence Engine, the product's differentiating spine; it has been refactored to a stable, hand-tuned state, so changes here require founder sign-off to avoid regressing the core scoring experience.
- `artifacts/lexy/src/pages/recruiter/candidates/index.tsx` — the primary, most-trafficked recruiter screen; likewise tuned to a stable state and owner-maintained, so changes require sign-off rather than ad-hoc edits.

These are *stable and high-stakes*, not broken — build around them with props and new components rather than modifying them directly.

- Always foreground the Intelligence Engine and internal-first sourcing in product/recruiter/testing guides.

3. The build gate (what actually has to pass)

esbuild is the gate, not `tsc`. There are ~300 pre-existing `tsc` errors (Express request-param typing noise). Use `tsc` as editor signal only. A change is "buildable" when:

```
pnpm --filter @workspace/api-server run build   # runs the two guards, then esbuild
```

succeeds. That `build` step runs, in order:

1. `check:no-console` — no stray `console.*` (use the pino logger).
2. `check:no-adverse-writes` — no ungoverned writes to adverse stages.
3. esbuild bundle to `dist/*.mjs`.

`check:deletion-cascade-drift` guards data-deletion safety; run it when touching deletion paths.

4. Testing standard

- Test runner is the **native Node runner via `tsx --test`** — no Jest/Vitest.
- Suites are scoped and fast (`test:sourcing`, `test:fairness`, `test:learned`, `test:similar`, `test:global`, `test:transcribe`). Run the relevant one(s) for your change.
- **`runTest` subagent is disabled in this environment — test manually.** Run the suite *and* exercise the actual flow.
- **Safety-critical logic must have a test** that encodes the *failure mode*, not just the happy path: fairness firewall, learned-scoring gates, the global-prior privacy boundary, provider hang/timeout. A test here is how we keep a guardrail from silently regressing (remember: esbuild won't catch a missing fairness import).

5. Code review standard

Major features get an **architect review** before being called done (the `code_review` skill / `architect`). Reviews look for, in priority:

1. **Security & tenant isolation** — scoping, auth-before-role, RLS-self-gating on `WITH CHECK(true)` policies, no foreign FK linkage.
2. **Silent-failure traps** — missing imports that fall back to a default, unhandled promise rejections, swallowed errors.
3. **Correctness of the change vs. blast radius** — did anything beyond the intended surface change?
4. **Honesty of outputs** — confidence caps, null vs zero, deterministic fallbacks.

Fix severe findings immediately; defer the rest into the backlog (Doc 9) with a reason.

6. Definition of Done

A change is done when **all** of these hold:

- [ ] esbuild build passes (incl. the two guard scripts).
- [ ] Tenant scoping verified for every data path touched.
- [ ] Relevant test suite passes; a new failure-mode test added if the change is safety-critical.
- [ ] Manually exercised the real flow (no `runTest` here).
- [ ] No new silent fallback to mocked/placeholder data.
- [ ] Non-obvious “why” recorded in `.agents/memory/` (with a `Why:` line).
- [ ] `replit.md` updated if architecture or a user-facing capability changed.
- [ ] Architect review passed for anything non-trivial; severe findings fixed.

7. Operational hygiene

- **Logging:** pino, structured. No `console.*`. Keep skip/error/timeout logs in a consistent shape so dashboards and alerts can parse them.
- **Schedulers:** there are ~16; a single scheduler-leader owns them. Don’t add a raw `setInterval` in a request path — add a scheduler or enqueue an `ai_job`.
- **Secrets:** never printed, never hand-edited in code. Managed through the environment/secrets tooling. Optional provider keys degrade gracefully when absent.
- **Migrations:** dev sync via `drizzle-kit push`; production via `docs/RUNBOOK_PROD_MIGRATIONS.md` (never `push` prod). Rebuild `@workspace/db` (`tsc -b`) after schema changes so downstream typecheck is sane.

8. Working with the memory system

`.agents/memory/MEMORY.md` is a one-line-per-topic index pointing at detail files. Before any non-trivial task: **skim it.** After: **update it** if you learned a durable lesson. Keep entries findable by

topic, never by task number. This file set is the curated, stable layer on top of that living memory.